ChatGPT

CLI、插件与 MCP 的本质与选型

执行摘要：CLI把能力压缩成可脚本化命令；插件把外部能力嵌入宿主运行时；MCP把“AI客户端↔工具/数据”的连接标准化。以浏览器与飞书为例，对比接口、触发、权限与无客户端方案，并给出选型建议。 [1]

假设与未指定项

你已明确“想理解概念并能做正确选型”，但有几处语境未指定，会直接影响“你说的 MCP 与实现路径”：

- MCP 的含义未明确：本文同时给出常见 MCP 含义，但在“飞书 MCP / openclaw MCP”语境下，优先按 **Model Context Protocol（模型上下文协议）** 解释，因为飞书开放平台与相关官方仓库已把 MCP 明确用于该协议与 AI 工具集成。 [2]
- openclaw 的版本与部署形态未给出：飞书官方的 openclaw 飞书/Lark 插件在 README 中注明需要 **openclaw ≥ 2026.2.26**，否则可能遇到问题。 [3]
- 是否需要代码示例未指定：本文以“架构与选型报告”为主，不默认给特定语言代码；若需要，可在后续按你的落地目标（例如“做一个飞书 MCP server”或“做一个浏览器插件 + 本地桥”）补最短可用样例。
- “飞书插件”指代可能混杂：在飞书生态里，“插件”既可能指 **多维表格（Base）扩展/插件**，也可能指“某个集成应用/机器人能力”，甚至指“飞书项目（Project）的插件”。本文会把“多维表格插件”与“openclaw 飞书官方插件”分开讲，并显式标注。 [4]

补充：MCP 的“常见含义”确实很多。维基百科的消歧页显示，MCP 可以指不同组织、技术名词等；简体/繁体条目都列出多种含义，其中也包含“Model Context Protocol”。 [5]

定义与本质

CLI 是什么与本质是什么

CLI（命令行界面）是**基于文本**的交互界面：用户（或程序/Agent）输入命令，系统通过 Shell/命令解释器把命令转换为进程调用与参数解析，从而驱动操作系统或应用完成动作。 [6]

从第一性原理看，CLI 的本质是：**把“能力”压缩成一组可组合、可脚本化、可自动化的“指令集合”**。当能力可以被表达为命令（动词）+ 参数（对象、范围、策略）+ 输出（结构化结果）时，CLI 就能成为最低摩擦的“执行入口”，尤其适合自动化与管道式组合（stdin/stdout、pipe、重定向）。 [7]

插件是什么与本质是什么

插件（plugin）通常指：**在不修改宿主核心代码的前提下**，以某种受控机制（接口、事件、脚本、扩展点）加载外部模块，实现功能扩展。插件化/微内核架构的典型思想是“核心系统负责生命周期与通用能力，具体功能由插件模块提供”。 [8]

从第一性原理看，插件的本质是：**把“可变需求”从“核心稳定域”剥离出去，变成可插拔的扩展域**。要做到这一点，宿主必须先回答三个问题：
1) 哪些能力必须保持稳定并长期兼容（核心）
2) 哪些点需要被扩展（扩展点/契约）
3) 扩展如何被隔离与授权（沙箱、权限、版本与风控） [9]

MCP 是什么与本质是什么

MCP 多种含义与典型场景

当你只说 “MCP”，它可能是多义缩写。一个实用的判别法是：看上下文是否出现 “AI/Agent/工具调用/服务器/协议”。若是，MCP 高概率指 Model Context Protocol。[10]

常见（且在技术圈更可能遇到的）几类 MCP 含义与场景示例：

- **Model Context Protocol（模型上下文协议）**：面向 AI 应用/Agent，把外部数据源、工具与工作流以标准方式连接起来；官方文档将其类比为 AI 应用的 “USB-C 接口”。[11]
- **Multi-Chip Package（多芯片封装）**：硬件/电子工程语境。[12]
- **Microsoft Certified Professional（微软认证专家）**等认证缩写：职业认证语境。[12]
- **Mod Coder Pack（Minecraft Coder Pack）**：游戏 Mod 开发工具链语境。[13]
- 其他组织/媒体条目：取决于行业（消歧页会列出）。[5]

在你这份需求里，MCP 的“本质”更应按 Model Context Protocol 理解

Model Context Protocol（本文后续默认所指的 MCP）在官方定义中是一个**开源标准**：用于把 AI 应用连接到外部系统，使 AI 客户端能连接数据源、工具与工作流。[14]

从第一性原理看，MCP 的本质是：**把“工具调用”从每家各做一套的私有集成，提升为“可发现（discoverable）+可授权（auth）+可约束（schema/permissions）+可复用（cross-client）”的标准接口层**。它解决的是“AI 客户端想用工具，但每个工具都要单独写适配、单独处理授权与参数结构”的系统性摩擦。[11]

飞书开放平台把 MCP 作为 AI 场景能力来提供，并强调基于 OpenAPI 做了面向 AI 的封装与优化（把传统接口调用升级为“工具化能力”，提升模型调用准确性与效果）。[15]

浏览器与飞书的落地形态

浏览器如何对应 CLI、插件、MCP

浏览器在工程上非常适合用来“对照理解三者差别”，因为它同时具备：可脚本化入口、可扩展运行时、严格沙箱与权限系统。

浏览器的 CLI（以 Chromium/Chrome 为代表）

- 典型形态：通过命令行参数/开关改变行为；或者以 Headless 模式在无可视 UI 的环境运行。Chromium 官方文档明确说明浏览器支持命令行开关来启用功能或改变默认行为；Chrome Headless 可在无人值守环境运行。[16]
- 本质映射：浏览器能力 → 命令参数化；适合自动化测试、抓取、批处理、CI。[17]

浏览器的插件（Extensions / WebExtensions）

- 典型形态：扩展通过 manifest 声明能力与权限；需要访问的 API/功能必须在 manifest 中声明 permissions。[18]
- 权限模型要点：扩展权限是显式声明与用户/浏览器侧控制的一部分；扩展默认受浏览器沙箱与 API 约束。[18]

浏览器里的 MCP（Model Context Protocol）怎么落地
浏览器本身不是 MCP 的“原生宿主”，但它能成为 MCP 生态的一部分，常见两条路径：

- 路径 A：**浏览器作为“远程 MCP 客户端”的 UI**（即：你在网页里用某个 MCP 客户端/AI 产品，它连接远程 MCP Server）。这通常要求 MCP Server 以网络方式暴露（例如 HTTP/SSE/WS 等），因为浏览器环境无法像桌面应用那样自由 spawn 本地进程来走 stdio。这个约束来自浏览器安全模型本身（网页无法直接运行本地可执行文件）。 [11]
- 路径 B：**浏览器扩展 + 本地桥接**：浏览器提供 Native Messaging，让扩展与本地原生应用通信；Chrome 文档明确说明会在单独进程启动 Native Messaging host，并用标准输入/输出与之通信。 [19]
- 这条路径在工程上很像“把 MCP server 放在本机进程里，然后用扩展把它接到浏览器 UI”。（注意：这里的 MCP 与 Native Messaging 都用到了 stdio 的思路，但它们是不同协议层；Native Messaging 是浏览器扩展↔本机应用的通道，MCP 是 AI 客户端↔工具的标准协议。） [20]

飞书如何对应 CLI、插件、MCP

飞书的 CLI（至少两类，容易混淆）

- “面向业务操作/Agent”的飞书 CLI：lark-cli（GitHub：larksuite/cli）自述为官方 CLI，面向人类与 AI Agents，覆盖消息、文档、多维表格、日历、邮箱、任务等，并强调三层架构（Shortcuts → API Commands → Raw API）与安全特性（例如注入防护、凭证存储）。 [21]
- “面向应用开发”的开发者工具 CLI：飞书开发者工具提供命令行（opdev/opdev-cli）用于项目管理、账号登录、预览与上传等开发流程。 [22]

飞书的插件（这里按“多维表格 Base 插件/扩展”最典型）

- 多维表格扩展（Base extensions）被描述为基于 Base 的扩展能力，可将自研产品嵌入 Base，并提供扩展点与插件市场/模板等生态要素。 [23]
- 多维表格自动化插件（automation extension）等属于“在飞书内嵌运行的扩展形态”，开发指南提到从创建插件应用、搭建环境、调试到发布安装的流程。 [24]

飞书的 MCP（OpenAPI MCP / 飞书 MCP Server）

- 飞书开放平台提供 MCP 能力，并把它定位为面向 AI 场景的工具化封装，用于提升模型调用准确性与效果。 [15]
- 官方 OpenAPI MCP 工具仓库（larksuite/lark-openapi-mcp）明确说明：它把飞书/Lark 开放平台 API 封装成 MCP tools，供 AI 助手直接调用以实现自动化（文档处理、会话管理、日程等），并处于 Beta。 [25]

openclaw 在飞书里的“插件/集成”位置

- 飞书官方出品的 openclaw 飞书/Lark 插件（larksuite/openclaw-lark）用于把 openclaw Agent 连接到飞书/Lark 工作区，能力覆盖消息、文档、多维表格、日历、任务等，并支持消息卡片交互、流式输出、权限策略与群组级配置。 [26]
- 该插件也强调风险：授权后 openclaw 会在授权范围内以你的用户身份行动，需谨慎使用，并建议作为私密助手使用、避免放进群聊以降低权限滥用与泄露风险。 [27]

关键对比表

对象/实现（示例）	接口类型	触发方式	权限模型	运行环境	是否需要客户端	典型用例
浏览器 CLI（Chromium/Chrome flags、Headless）	命令行参数/进程启动参数	人/脚本/CI 调用可执行文件	继承 OS 用户权限；浏览器自身安全策略仍生效	本机或服务器（可无头）	不需要“图形客户端”；但需要运行浏览器进程	自动化测试、爬取/截图/PDF、批处理渲染 [28]
浏览器插件（Chrome Extensions / WebExtensions）	扩展 API + manifest	点击、页面事件、浏览器事件	manifest 声明 permissions；受沙箱限制	浏览器扩展运行时	需要浏览器（可本地/企业环境）	页面增强、表单填充、内容提取、企业策略扩展 [18]
浏览器插件 + 本地桥（Native Messaging）	扩展 API + 本地进程 stdin/stdout	插件触发→本地 host 执行	扩展需 nativeMessaging 权限；host 清单限制可连接扩展	本机（必须安装本地 host）	需要浏览器 + 本机可执行程序	密码管理器、硬件访问、把本地工具能力安全暴露给扩展 [19]
飞书多维表格插件/扩展（Base extensions）	飞书内嵌扩展点 + SDK	用户在 Base 内操作/触发	依赖应用权限与扩展点授权；进入飞书生态权限体系	飞书客户端/网页内嵌运行	通常需要飞书客户端/网页端承载 UI	在表/仪表盘内做自定义视图、自动化、业务小工具 [29]

对象/实现（示例）	接口类型	触发方式	权限模型	运行环境	是否需要客户端	典型用例
openclaw 飞书官方插件（openclaw-lark）	“渠道/机器人”集成 + 飞书开放平台 API	在飞书里与 bot 对话、卡片按钮确认等	授权后以用户身份在授权范围内操作；提供权限策略/群配置；存在提示注入风险	openclaw 运行时（本地/服务器）+ 飞书工作区	需要飞书作为交互入口；openclaw 可自托管	把消息/文档/日程等上下文喂给 Agent，并在飞书内闭环执行动作 [30]
飞书 CLI（lark-cli）	本地 CLI + Skills（给 Agent 用的结构化工具描述）	终端命令；AI 编程工具/Agent 调用命令	通过应用与登录授权；强调注入防护与凭证存储；三层架构	本机或可访问飞书的运行环境	不需要飞书客户端本体；需要终端环境	在脚本/IDE/Agent 内批量操作飞书资源（消息、文档、Base、日历等） [21]
飞书 OpenAPI MCP（lark-openapi-mcp / 飞书 MCP）	MCP Server（把 OpenAPI 封装成 Tools）	MCP 客户端（Claude/Cursor 等）发起工具调用	App ID/Secret；可选 OAuth 登录获取 user access token；可选 token-mode；Beta 且有功能限制提示	本地（npx 启动）或可部署为服务（取决于传输模式/实现）	不需要飞书客户端；需要 MCP 客户端	让 MCP 兼容 AI 工具直接调用飞书能力；跨客户端复用工具定义 [31]
飞书开发者工具 CLI（opdev/opdev-cli）	开发工具链 CLI	开发者执行 login/preview/upload 等	开发者账号/应用开发权限	本机开发环境	不面向普通业务用户；面向开发者	小程序/扩展等项目开发、调试、上传发布 [22]

openclaw 接入飞书插件、飞书 CLI 与飞书 MCP 的重叠与差异

功能是否重叠

如果只看“能做什么事”，三者确实**高度重叠**：

- openclaw 飞书官方插件宣称可读写消息、文档、多维表格、日历、任务等。[32]
- lark-cli README 列出同类业务域，并强调 200+ 命令、技能与三层架构覆盖。[21]
- 飞书 OpenAPI MCP 工具把开放平台 API 封装成 MCP tools，用于文档处理、会话管理、日程等自动化。[33]

因此，差异主要不在“能力集合”，而在“你到底想把能力交给谁、在什么环境里、以什么授权与体验来执行”。

差异的第一性原理拆解

把这三者拆开看，它们分别回答的是不同问题：

openclaw + 飞书插件回答：
“我要让 AI 在飞书里成为一个可对话的执行体，并把飞书当作上下文与交互入口。”

- 触发与体验：消息对话、卡片交互、流式回复；更像“把 Agent 放进协作现场”。[34]
- 权限与风险：插件明确提示授权后 Agent 以你的身份在授权范围内行动，并强调提示注入/不可预测执行风险，建议私用、避免进群。[27]
- 工程含义：你需要维护 openclaw 运行时与安全边界；官方插件也提出版本约束（≥2026.2.26）。[3]

飞书 CLI（lark-cli）回答：
“我要把飞书能力变成可脚本化的命令，让人或 AI 编程工具直接在终端执行。”

- 强项：可组合、可测试、易集成到 CI/脚本/IDE；README 强调 Agent-Native 的 Skills、结构化输出、注入防护与凭证存储。[21]
- 体验代价：对终端与本机环境有要求；更偏“工程执行入口”，不是“业务现场交互入口”。[35]

飞书 MCP（OpenAPI MCP / lark-openapi-mcp）回答：
“我要让任意 MCP 兼容的 AI 客户端，以统一协议发现并调用飞书工具。”

- 强项：跨客户端复用（Cursor/Claude 等）；工具以 schema 形式暴露，更利于模型理解与参数约束。[36]
- 权限形态：仓库 README 给出用 App ID/Secret 启动、可选 OAuth 登录与 token-mode 的说明，强调 user_access_token 更适合访问用户私有资源。[25]
- 已知限制：README 明确提示某些能力（如文件上传/下载、云文档直接编辑）尚不支持，并标注 Beta。[25]

你应该在什么场景选哪一个

以“最短闭环”为准则，可以这样选：

- **你希望“人在飞书里说一句，事情就在飞书里办完”**：优先 openclaw + 飞书官方插件。它把“上下文、触发、反馈、审批确认”都放进飞书对话/卡片里，体验闭环短。[34]
- **你希望“在本地/服务器脚本里批量跑飞书自动化（定时、CI、运维）”**：优先 lark-cli（或直接 OpenAPI），因为 CLI 的可测试性与可组合性更强。[35]

- **你希望“让 Cursor/Claude Desktop 这类 AI 客户端开箱即用地调用飞书能力”**：优先飞书 OpenAPI MCP（lark-openapi-mcp），因为它是面向 MCP 客户端的标准接口形态。 [37]

补充一个“组合方案”，用于解释你提出的“openclaw、CLI、MCP 是否只是不同前端”：
openclaw 本身也支持以 MCP server 形态对外暴露会话与工具：其文档说明 `openclaw mcp serve` 会启动一个 stdio MCP server，由 MCP client 拉起该进程；bridge 再通过 WebSocket 连接 OpenClaw Gateway，把“渠道会话”暴露给 MCP。 [38]
这意味着：如果你已经把飞书作为 openclaw 的一个渠道接入，那么其它 MCP 客户端可以“通过 openclaw 统一入口”间接触达飞书会话——但这条链路更长、运维更重，只有在你需要“多渠道统一 + 统一安全策略/审批”时才划算。 [39]

可改造性与无客户端实现

是否每个软件都能改造成 CLI/插件/MCP

结论先行：**理论上“多数软件能力”都能被包装成 CLI/MCP，但“把软件改造成可插拔插件宿主”要求更高，且不总值得**。原因在于三者对“宿主改造深度”的要求不同。

改造成 CLI：门槛最低，但前提是“能力可参数化”

- 必要条件：存在稳定的“动作集合”（CRUD、查询、触发流程等），能被表达为命令 + 参数；输出能结构化（JSON/表格）用于下游自动化；错误需要可恢复/可诊断。CLI 作为文本接口，本来就服务于自动化与脚本化。 [6]
- 常见障碍：能力强依赖 GUI 状态（拖拽、所见即所得编辑、复杂选区）；若用 UI 自动化来“伪 CLI”，通常脆弱、维护成本高（DOM/控件变化就失效）。这类情况下更应先抽出 API/服务层。

改造成插件系统：门槛最高，因为你要提供“扩展点契约 + 生命周期 + 隔离/权限”

- 必要条件：宿主愿意长期维护扩展 API 的兼容性；能做插件隔离（沙箱/权限/资源限制）；能做版本管理与升级策略。插件化/微内核架构强调核心与插件分工，宿主至少要提供插件管理与最小接口集合。 [8]
- 常见障碍：安全边界难做（插件可以读/改什么数据）、生态治理难做（插件质量、兼容、审核）、以及“可扩展点”设计不清导致核心被插件侵蚀，最终反而更难维护。

改造成 MCP：通常“最易在外围实现”，因为你可以不改宿主，只包一层

- 必要条件：底层必须已经有 API/SDK/CLI 可调用；你能把这些动作映射成 MCP tools，并定义清晰 schema（输入/输出结构）。MCP 的目标正是标准化 AI 客户端与外部系统连接。 [11]
- 常见障碍：权限与审计（AI 何时能调用哪些工具）、提示注入导致的越权链路、以及工具组合引发的数据外泄风险。openclaw 官方插件对提示注入与不可预测执行的风险提示，反映了这类系统的真实安全挑战。 [27]

无客户端场景：可行性与何时仍需客户端

这里的“无需客户端”需要先澄清：通常指“无需安装桌面/移动端应用”，而不是“完全不需要任何可执行体”。在工程上，至少需要一个运行环境来发 HTTP 请求、接收事件、执行逻辑（服务器、函数计算、容器都算）。

飞书侧：大量能力可以做到“无客户端”闭环

- 最轻量：自定义机器人 webhook。飞书文档描述其为群聊内无需管理员审核即可使用的消息推送工具，通过 webhook URL 推送文本/富文本/图片等。[40]
- 更强：应用机器人 + 开放平台 API + 事件订阅/回调。飞书开放平台提供事件订阅用于及时响应变更事件；也区分“回调（同步，需立即响应）”与“事件（不要求返回）”。[41]

openclaw 接入飞书时的一个关键点
openclaw 文档对“飞书渠道插件”的描述提到：通过平台的 WebSocket 事件订阅把 openclaw 连接到飞书/Lark 机器人，因此无需暴露公共 webhook URL 来接收消息。[42]
这说明“无客户端”并不等于“无长连接/无服务端”；它只是把交互入口从客户端转移到服务端/网关，并在安全与网络暴露上做取舍。

什么时候仍然需要客户端支持（或至少需要“本地执行体”）

- 需要访问本地资源：本地文件系统、浏览器本地凭证、企业内网只在终端可达的系统等。此时通常需要 CLI、桌面 MCP 客户端、或浏览器扩展 + Native Messaging 这类本地桥。[43]
- 需要 UI 级能力：例如在浏览器内对真实页面进行交互，往往需要运行浏览器（可以是 Headless，但仍是一个“客户端进程”）。[17]
- 需要“用户上下文绑定”：例如必须以用户身份在其会话中完成某些操作，常见做法是 OAuth 登录/授权（飞书 MCP 工具 README 也强调 OAuth 与 token-mode 选择）。[25]

本质类比与实践建议

“本质上都是实现功能、只是不同前端”这句话对吗

这句话**一半对、一半会误导**。

对的部分：
无论网页、安卓、Windows 客户端、CLI、Bot，本质上都是“某种客户端/交互面”去调用后端能力（API、数据、工作流）。当底层能力抽象得足够好（API 清晰、权限可控），你确实能在不同前端之间共享绝大多数能力实现。[44]

容易误导的部分：

- **插件**更像“把前端/逻辑嵌到宿主内部运行”，它不只是前端，而是“宿主运行时的一部分”。[45]
- **MCP**本质更接近“协议/适配层”，它把工具以标准方式暴露给 AI 客户端；它不等同于某个 UI 前端，而是让“不同 AI 前端”共享同一套工具定义与调用逻辑。[11]

可以用一句更准确的类比来替代：

- API/能力层：你真正能做的事（读写文档、发消息、建日程）
- 适配层：CLI 包装器、MCP server、SDK
- 交互层：飞书聊天/卡片、浏览器扩展 UI、终端、人机对话框
  openclaw、飞书 CLI、飞书 MCP 的差别主要落在“适配层 + 交互层如何组合”。[46]

选型建议：CLI vs 插件 vs MCP vs API vs Bot

下面用一个“以目标为中心”的决策图来直接指导实现方式选择。

```
flowchart TD
  A[你的目标是什么] --> B{触发入口在哪里}
  B -->|在飞书对话/群里| C[Bot/渠道集成]
  B -->|在终端/脚本/CI里| D[CLI 或直接 API]
  B -->|在AI客户端如IDE/桌面助手里| E[MCP]
  C --> C1{是否只需单向通知}
  C1 -->|是| C2[Webhook自定义机器人]
  C1 -->|否，需读写业务数据| C3[应用机器人 + OpenAPI/事件订阅]
  C3 --> C4{是否需要“Agent闭环+卡片交互+审批”}
  C4 -->|是| C5[openclaw + 飞书官方插件]
  C4 -->|否| C6[直接Bot工作流]
  E --> E1{是否希望跨多个AI客户端复用}
  E1 -->|是| E2[OpenAPI MCP/自建MCP server]
  E1 -->|否| E3[CLI skills 或 API SDK]
```

图中每个分支背后的关键“为什么”：

- **只做通知**：Webhook 机器人最快，因为它的目标就是“外部系统推消息进群”，文档明确其通过 webhook 地址完成消息推送且无需管理员审核（在群内范围）。 40
- **要读写业务对象**：应用机器人/自建应用 + OpenAPI/事件订阅更稳，因为你需要权限、鉴权与回调链路。 47
- **要在飞书里做 Agent 闭环**：openclaw 官方插件提供卡片交互、流式输出、权限策略与群配置，但也带来提示注入与用户身份操作风险，需要更严的权限与风控。 34
- **要在 AI 编程工具里“工具即插即用”**：飞书 OpenAPI MCP 把 OpenAPI 封装为 MCP tools，适配 MCP 客户端的调用路径；官方工具同时给出 OAuth 与 token-mode 选择建议。 37
- **要做工程自动化**：lark-cli 把能力变成命令与 skills，优势在可测试、可组合、易集成到脚本，并强调注入防护与凭证存储。 21

设计要点与测试要点

认证与权限
最小权限是底线：无论用 CLI、MCP 还是插件，核心都是“把谁的身份、在什么范围内、允许做哪些动作”表达清楚。openclaw 飞书插件直接提醒“授权后以用户身份在授权范围内行动”，这是选型时必须纳入威胁模型的信号。 48
飞书 OpenAPI MCP README 也提示在启用 OAuth 时，建议显式设置 token-mode 为 user_access_token，以更贴近用户资源访问语义，并避免自动回退导致权限不足或访问不到私有数据。 25

可扩展性与迁移策略（最短路径）
如果你未来可能同时支持 CLI、MCP、插件，最省返工的路径通常是：
先把“能力层”沉到 API/服务（或稳定 SDK）→ 再提供 CLI/MCP 适配 → 最后在需要强体验的地方做插件/卡片 UI。MCP 的价值在于让多个 AI 前端共享同一套工具定义，而 CLI 的价值在于重用同一套能力做脚本化与可测试自动化。 49

测试要点（尤其涉及 Agent）

- 权限测试：最小权限下逐条工具调用的成功/失败边界（确保失败可解释、可引导补授权）。 50
- 安全测试：提示注入场景下是否可能诱导越权与数据泄漏（openclaw 插件明确把它列为风险）。 48
- 可观测性：记录“谁在何时调用了什么工具、对什么资源做了什么改动”，否则出了问题无法审计与回滚（在协作系统里这是高频真实需求）。
- 可靠性：幂等、重试、超时、速率限制与部分失败处理（尤其是批量写入、跨多个对象操作）。

主要参考来源

本文优先使用官方文档、飞书开放平台与官方仓库，其次选取权威技术文档/博客作为补充：

- Amazon Web Services [51] 对 CLI 的定义与工作方式说明（中文/繁中页面一致）。 [6]
- Chromium/Chrome 官方文档：命令行开关与 Headless 模式。 [28]
- Chrome 扩展官方文档：manifest 权限声明、Native Messaging（单独进程 + stdin/stdout）。 [52]
- Mozilla [53] MDN：WebExtensions Native Messaging 的权限与清单约束。 [54]
- MCP 官方文档与权威解读：Model Context Protocol 的定义与价值。 [55]
- 飞书官方开源：lark-cli（larksuite/cli）。 [21]
- 飞书官方开源：OpenAPI MCP（larksuite/lark-openapi-mcp）与飞书开放平台 MCP能力说明（检索摘要）。 [33]
- openclaw 与其 MCP/gateway 文档：Gateway WebSocket 控制平面，`openclaw mcp serve` 的 stdio MCP server 机制。 [56]
- 飞书官方：openclaw 飞书官方插件文章与仓库（openclaw-lark）。 [57]
- 飞书开放平台：自定义机器人 webhook 与事件订阅/回调相关文档（检索摘要）。 [58]

---

[1] [6] 什么是CLI? - 命令行界面详解
https://aws.amazon.com/cn/what-is/cli/?utm_source=chatgpt.com

[2] [15] MCP 概述- MCP - 开发文档- 飞书开放平台
https://open.feishu.cn/document/uAjLw4CM/ukTMukTMukTM/mcp_integration/mcp_introduction?lang=zh-CN&utm_source=chatgpt.com

[3] [26] [27] [30] [32] [34] [46] [48] GitHub - larksuite/openclaw-lark: 飞书官方出品的 OpenClaw 飞书/Lark Channel 插件 · GitHub
https://github.com/larksuite/openclaw-lark

[4] 多维表格开放能力简介
https://open.larksuite.com/document/uAjLw4CM/uYjL24iN/base-extensions/base-extension-introduction?lang=zh-CN&utm_source=chatgpt.com

[5] MCP
https://en.wikipedia.org/wiki/MCP?utm_source=chatgpt.com

[7] [44] 什麼是CLI? – 命令行介面說明
https://aws.amazon.com/tw/what-is/cli/?utm_source=chatgpt.com

[8] [9] [45] 最经典的两种软件架构模式
https://tech.youzan.com/zui-jing-dian-de-liang-chong-ruan-jian-jia-gou-mo-shi/?utm_source=chatgpt.com

[10] [12] MCP - 維基百科，自由的百科全書
https://zh.wikipedia.org/zh-tw/MCP?utm_source=chatgpt.com

[11] [14] [36] [49] [55] What is the Model Context Protocol (MCP)? - Model Context Protocol
https://modelcontextprotocol.io/docs/getting-started/intro

[13] MCP-Reborn is an MCP (Mod Coder Pack) for Minecraft ...
https://github.com/Hexeption/MCP-Reborn?utm_source=chatgpt.com

[16] [28] Run Chromium with command-line switches
https://www.chromium.org/developers/how-tos/run-chromium-with-flags/?utm_source=chatgpt.com

17 Chrome Headless mode | Chromium
https://developer.chrome.com/docs/chromium/headless?utm_source=chatgpt.com

18 52 Declare permissions | Chrome Extensions
https://developer.chrome.com/docs/extensions/develop/concepts/declare-permissions?utm_source=chatgpt.com

19 20 43 原生消息传递
https://developer.chrome.com/docs/extensions/develop/concepts/native-messaging?hl=zh-cn&utm_source=chatgpt.com

21 35 53 GitHub - larksuite/cli: The official Lark/Feishu CLI tool, maintained by the larksuite team — built for humans and AI Agents. Covers core business domains including Messenger, Docs, Base, Sheets, Calendar, Mail, Tasks, Meetings, and more, with 200+ commands and 19 AI Agent Skills. · GitHub
https://github.com/larksuite/cli

22 Feishu Developer Tool - Command Line - 开发指南
https://open.feishu.cn/document/uYjL24iN/uEzMzUjLxMzM14SMzMTN/ide-with-commands?utm_source=chatgpt.com

23 29 Base Extension introduction - Developer Guides
https://open.larksuite.com/document/uAjLw4CM/uYjL24iN/base-extensions/base-extension-introduction?from=home&utm_source=chatgpt.com

24 51 多维表格自动化插件开发指南
https://open.feishu.cn/document/base-extensions/base-automation-extension-development-guide?lang=zh-CN&utm_source=chatgpt.com

25 31 33 37 50 GitHub - larksuite/lark-openapi-mcp: 飞书/Lark官方 OpenAPI MCP · GitHub
https://github.com/larksuite/lark-openapi-mcp

38 39 mcp - OpenClaw
https://docs.openclaw.ai/cli/mcp

40 58 自定义机器人使用指南- 开发文档- 飞书开放平台
https://open.feishu.cn/document/client-docs/bot-v3/add-custom-bot?lang=zh-CN&utm_source=chatgpt.com

41 47 Event overview - Documentation - Feishu Open Platform - Lark
https://open.feishu.cn/document/server-docs/event-subscription-guide/overview?utm_source=chatgpt.com

42 飞书
https://docs.openclaw.ai/zh-CN/channels/feishu?utm_source=chatgpt.com

54 与本地应用通信 - MDN Web Docs
https://developer.mozilla.org/zh-CN/docs/Mozilla/Add-ons/WebExtensions/Native_messaging?utm_source=chatgpt.com

56 Gateway 网关协议 - OpenClaw
https://docs.openclaw.ai/zh-CN/gateway/protocol

57 OpenClaw飞书官方插件上线｜一文讲清功能、安装更新教程 ...
https://www.feishu.cn/content/article/7613711414611463386?utm_source=chatgpt.com